



CSA0610 – DESIGN AND ANALYSIS OF ALGORITHM

1. Check the palindrome in a array:

```
topic 1 program 1 -palindrome string in a array.py - C:/Users/HP/OneDrive/Desktop/DAA lab programs/topic...
File Edit Format Run Options Window Help

def first_palindrome(words):
    # Function to check if a string is palindrome
    def is_palindrome(s):
        return s == s[::-1]

    # Iterate through the list of words
    for word in words:
        if is_palindrome(word):
            return word
    return ""

# Example 1
words1 = ["abc", "car", "ada", "cool", "malayalam"]
print(first_palindrome(words1)) # Output: "ada"

# Example 2
words2 = ["notapalindrome", "racecar"]
print(first_palindrome(words2)) # Output: "racecar"

# Example 4
words3 = ["notapalindrome", "malayalam"]
print(first_palindrome(words3))

# Example 3 (no palindrome)
words3 = ["hello", "world"]
print(first_palindrome(words3)) # Output: ""

Ln 21 Col 6
```

```
IDLE Shell 3.13.5
File Edit Shell Debug Options Window Help

Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> = RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programs/topic 1 -palindrome string in a array.py
ada
racecar
malayalam
>>>

Ln 9 Col 0
```

2. Count the common elements in a array

```
program2 count num 1 and num 2.py - C:/Users/HP/OneDrive/Desktop/DAA lab programs/program2 count n...
File Edit Format Run Options Window Help

def count_common_elements(nums1, nums2):
    set1 = set(nums1)
    set2 = set(nums2)

    answer1 = sum(1 for x in nums1 if x in set2)
    answer2 = sum(1 for x in nums2 if x in set1)

    return [answer1, answer2]

# Input
nums1 = [2,3,2,3,4,5]
nums2 = [2,3,3,4,4,5,5]

result = count_common_elements(nums1, nums2)

# Output
print("count of the common element in array", result)

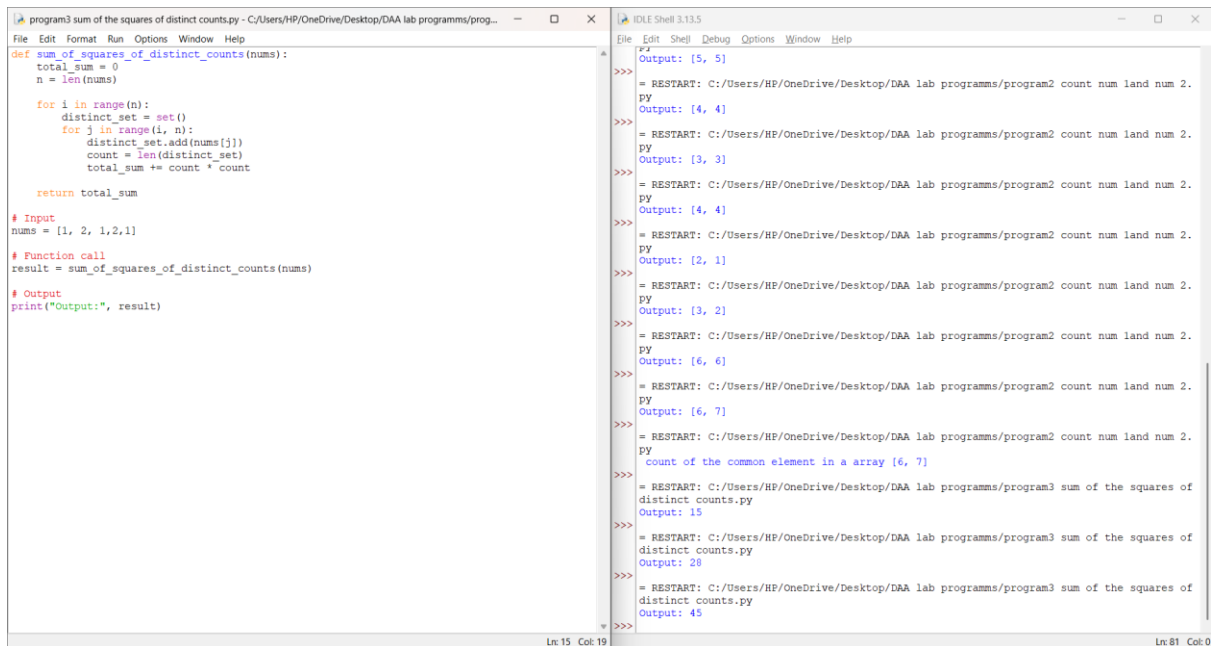
Ln 20 Col 40
```

```
IDLE Shell 3.13.5
File Edit Shell Debug Options Window Help

>>> Output: [4, 5]
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programs/program2 count num land num 2.
py
Output: [4, 5]
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programs/program2 count num land num 2.
py
Output: [3, 4]
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programs/program2 count num land num 2.
py
Output: [5, 5]
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programs/program2 count num land num 2.
py
Output: [4, 4]
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programs/program2 count num land num 2.
py
Output: [3, 3]
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programs/program2 count num land num 2.
py
Output: [4, 4]
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programs/program2 count num land num 2.
py
Output: [2, 1]
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programs/program2 count num land num 2.
py
Output: [3, 2]
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programs/program2 count num land num 2.
py
Output: [6, 6]
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programs/program2 count num land num 2.
py
Output: [6, 7]
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programs/program2 count num land num 2.
py
count of the common element in a array [6, 7]
>>>

Ln 72 Col 0
```

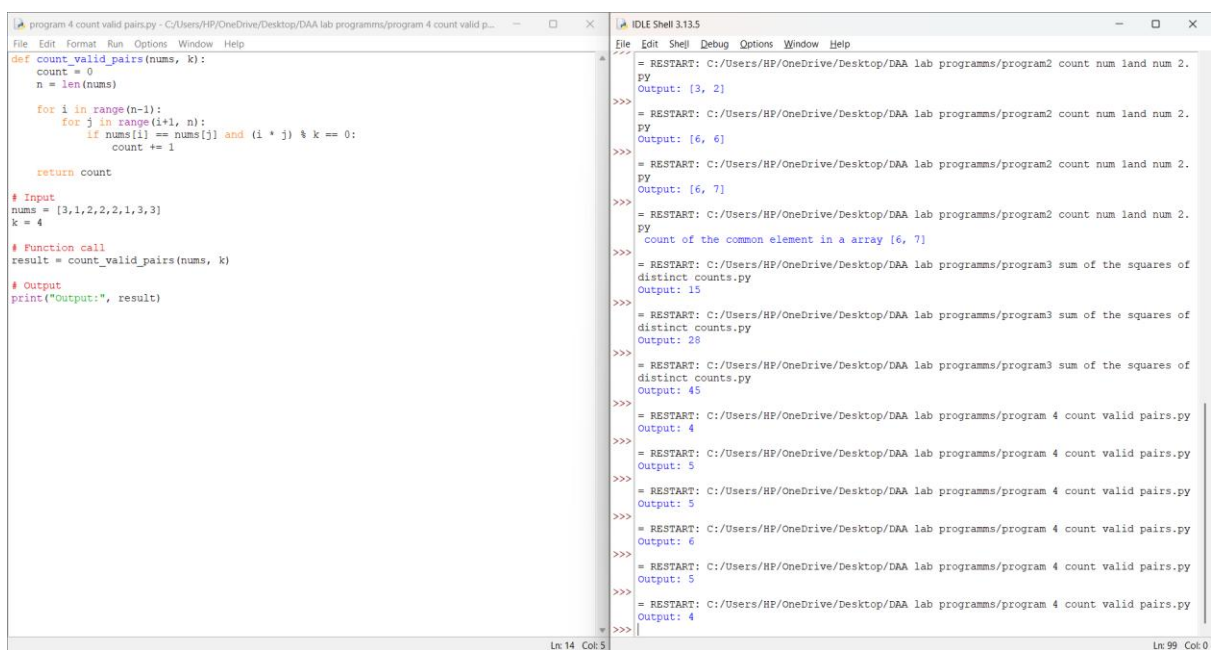
3. sum of the squares of the distinct counts:



```
program3 sum of the squares of distinct counts.py - C:/Users/HP/OneDrive/Desktop/DAA lab programmes/program3 sum of the squares of distinct counts.py
def sum_of_squares_of_distinct_counts(nums):
    total_sum = 0
    n = len(nums)
    for i in range(n):
        distinct_set = set()
        for j in range(i, n):
            distinct_set.add(nums[j])
            count = len(distinct_set)
            total_sum += count * count
    return total_sum
# Input
nums = [1, 2, 1, 2, 1]
# Function call
result = sum_of_squares_of_distinct_counts(nums)
# Output
print("Output:", result)
```

```
IDLE Shell 3.13.5
Output: [5, 5]
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programmes/program2 count num land num 2.
PY
Output: [4, 4]
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programmes/program2 count num land num 2.
PY
Output: [3, 3]
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programmes/program2 count num land num 2.
PY
Output: [4, 4]
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programmes/program2 count num land num 2.
PY
Output: [2, 1]
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programmes/program2 count num land num 2.
PY
Output: [3, 2]
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programmes/program2 count num land num 2.
PY
Output: [6, 6]
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programmes/program2 count num land num 2.
PY
Output: [6, 7]
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programmes/program2 count num land num 2.
PY
count of the common element in a array [6, 7]
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programmes/program3 sum of the squares of
distinct counts.py
Output: 15
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programmes/program3 sum of the squares of
distinct counts.py
Output: 28
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programmes/program3 sum of the squares of
distinct counts.py
Output: 45
>>>
```

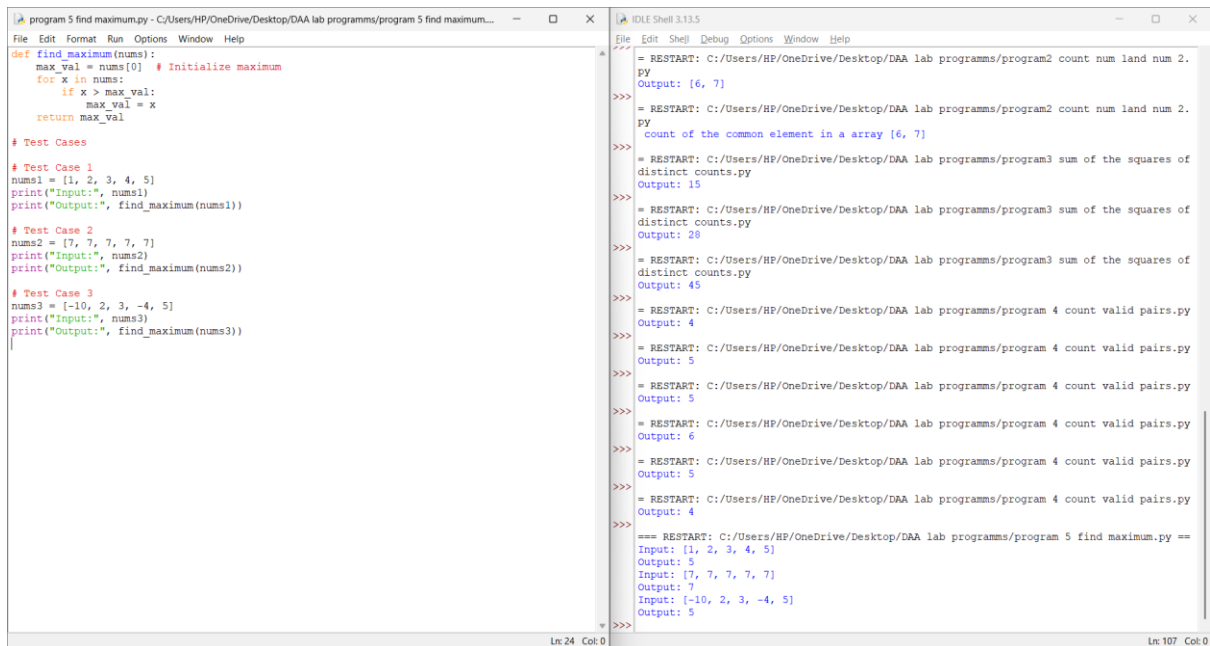
4. Count the valid pairs:



```
program 4 count valid pairs.py - C:/Users/HP/OneDrive/Desktop/DAA lab programmes/program 4 count valid p...
def count_valid_pairs(nums, k):
    count = 0
    n = len(nums)
    for i in range(n-1):
        for j in range(i+1, n):
            if nums[i] == nums[j] and (i * j) % k == 0:
                count += 1
    return count
# Input
nums = [3,1,2,2,2,1,3,3]
k = 4
# Function call
result = count_valid_pairs(nums, k)
# Output
print("Output:", result)
```

```
IDLE Shell 3.13.5
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programmes/program2 count num land num 2.
PY
Output: [3, 2]
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programmes/program2 count num land num 2.
PY
Output: [6, 6]
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programmes/program2 count num land num 2.
PY
Output: [6, 7]
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programmes/program2 count num land num 2.
PY
count of the common element in a array [6, 7]
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programmes/program3 sum of the squares of
distinct counts.py
Output: 15
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programmes/program3 sum of the squares of
distinct counts.py
Output: 28
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programmes/program3 sum of the squares of
distinct counts.py
Output: 45
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programmes/program4 count valid pairs.py
Output: 4
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programmes/program4 count valid pairs.py
Output: 5
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programmes/program4 count valid pairs.py
Output: 5
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programmes/program4 count valid pairs.py
Output: 6
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programmes/program4 count valid pairs.py
Output: 5
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programmes/program4 count valid pairs.py
Output: 4
>>>
```

5. Find the maximum element in the array:



```
program 5 find_maximum.py - C:/Users/HP/OneDrive/Desktop/DAA lab programs/program 5 find_maximum.py
def find_maximum(nums):
    max_val = nums[0] # Initialize maximum
    for x in nums:
        if x > max_val:
            max_val = x
    return max_val

# Test Cases

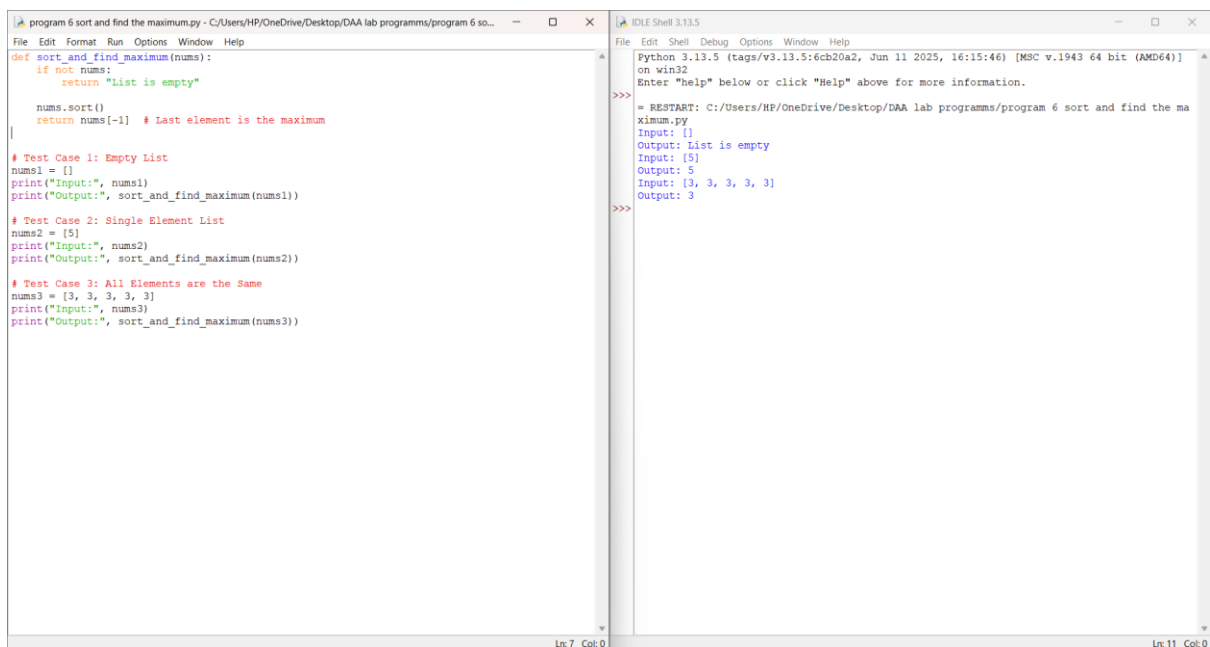
# Test Case 1
nums1 = [1, 2, 3, 4, 5]
print("Input:", nums1)
print("Output:", find_maximum(nums1))

# Test Case 2
nums2 = [7, 7, 7, 7, 7]
print("Input:", nums2)
print("Output:", find_maximum(nums2))

# Test Case 3
nums3 = [-10, 2, 3, -4, 5]
print("Input:", nums3)
print("Output:", find_maximum(nums3))

IDLE Shell 3.13.5
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programs/program2 count num land num 2.
py
Output: [6, 7]
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programs/program2 count num land num 2.
py
count of the common element in a array [6, 7]
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programs/program3 sum of the squares of
distinct counts.py
Output: 15
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programs/program3 sum of the squares of
distinct counts.py
Output: 28
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programs/program3 sum of the squares of
distinct counts.py
Output: 45
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programs/program 4 count valid pairs.py
Output: 4
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programs/program 4 count valid pairs.py
Output: 5
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programs/program 4 count valid pairs.py
Output: 5
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programs/program 4 count valid pairs.py
Output: 6
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programs/program 4 count valid pairs.py
Output: 5
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programs/program 4 count valid pairs.py
Output: 4
>>>
== RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programs/program 5 find_maximum.py ==
Input: [1, 2, 3, 4, 5]
Output: 5
Input: [7, 7, 7, 7, 7]
Output: 7
Input: [-10, 2, 3, -4, 5]
Output: 5
>>>
```

6. To sort the array and find the maximum.



```
program 6 sort and find the maximum.py - C:/Users/HP/OneDrive/Desktop/DAA lab programs/program 6 so...
def sort_and_find_maximum(nums):
    if not nums:
        return "List is empty"
    nums.sort()
    return nums[-1] # Last element is the maximum

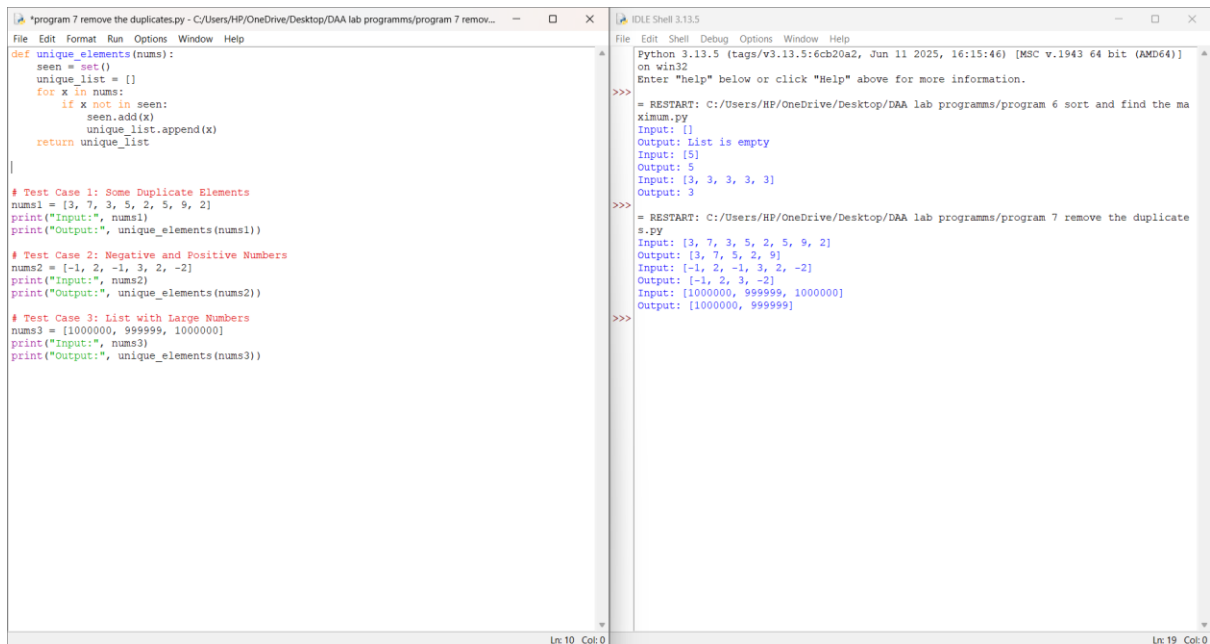
# Test Case 1: Empty List
nums1 = []
print("Input:", nums1)
print("Output:", sort_and_find_maximum(nums1))

# Test Case 2: Single Element List
nums2 = [5]
print("Input:", nums2)
print("Output:", sort_and_find_maximum(nums2))

# Test Case 3: All Elements are the Same
nums3 = [3, 3, 3, 3]
print("Input:", nums3)
print("Output:", sort_and_find_maximum(nums3))

IDLE Shell 3.13.5
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programs/program 6 sort and find the ma
ximum.py
Input: []
Output: List is empty
Input: [5]
Output: 5
Input: [3, 3, 3, 3]
Output: 3
>>>
```

7. Remove the duplicates:



```
File Edit Format Run Options Window Help
def unique_elements(nums):
    seen = set()
    unique_list = []
    for x in nums:
        if x not in seen:
            seen.add(x)
            unique_list.append(x)
    return unique_list

# Test Case 1: Some Duplicate Elements
nums1 = [3, 7, 3, 5, 2, 5, 9, 2]
print("Input:", nums1)
print("Output:", unique_elements(nums1))

# Test Case 2: Negative and Positive Numbers
nums2 = [-1, 2, -1, 3, 2, -2]
print("Input:", nums2)
print("Output:", unique_elements(nums2))

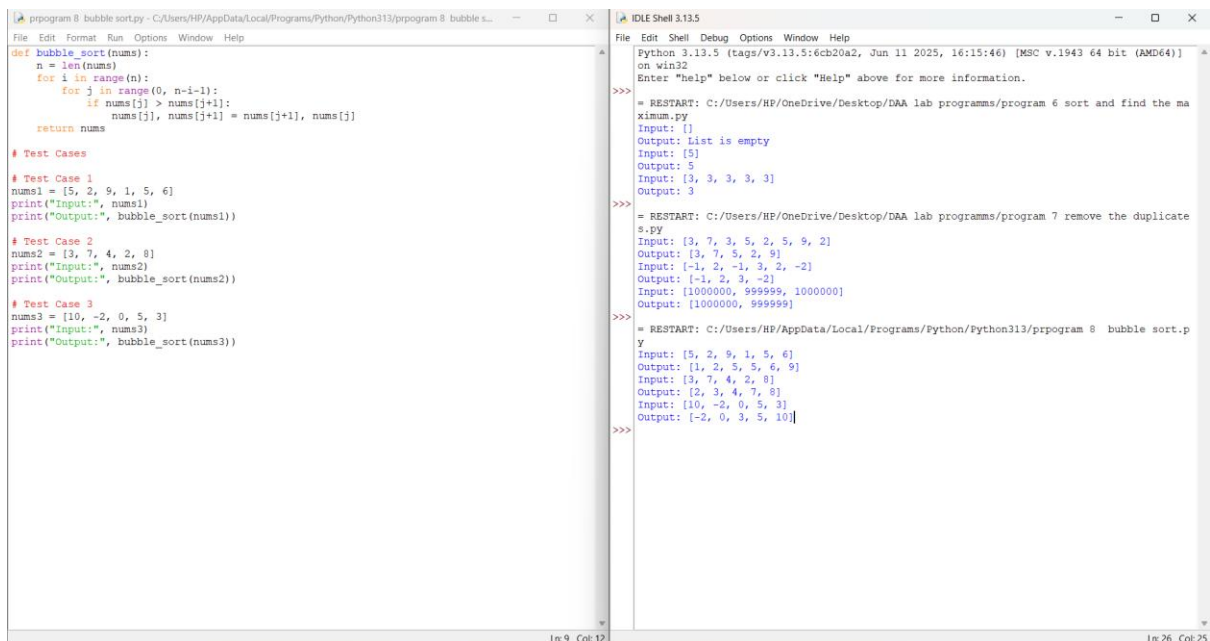
# Test Case 3: List with Large Numbers
nums3 = [1000000, 999999, 1000000]
print("Input:", nums3)
print("Output:", unique_elements(nums3))
```

```
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.

>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programmes/program 6 sort and find the maximum.py
Input: []
Output: List is empty
Input: [5]
Output: 5
Input: [3, 3, 3, 3]
Output: 3

>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programmes/program 7 remove the duplicate s.py
Input: [3, 7, 3, 5, 2, 5, 9, 2]
Output: [3, 7, 5, 2, 9]
Input: [-1, 2, -1, 3, 2, -2]
Output: [-1, 2, 3, -2]
Input: [1000000, 999999, 1000000]
Output: [1000000, 999999]
```

8. Sort the array by using bubble sort method:



```
File Edit Format Run Options Window Help
def bubble_sort(nums):
    n = len(nums)
    for i in range(n):
        for j in range(0, n-i-1):
            if nums[j] > nums[j+1]:
                nums[j], nums[j+1] = nums[j+1], nums[j]
    return nums

# Test Cases

# Test Case 1
nums1 = [5, 2, 9, 1, 5, 6]
print("Input:", nums1)
print("Output:", bubble_sort(nums1))

# Test Case 2
nums2 = [3, 7, 4, 2, 8]
print("Input:", nums2)
print("Output:", bubble_sort(nums2))

# Test Case 3
nums3 = [10, -2, 0, 5, 3]
print("Input:", nums3)
print("Output:", bubble_sort(nums3))
```

```
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.

>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programmes/program 6 sort and find the maximum.py
Input: []
Output: List is empty
Input: [5]
Output: 5
Input: [3, 3, 3, 3]
Output: 3

>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programmes/program 7 remove the duplicate s.py
Input: [3, 7, 3, 5, 2, 5, 9, 2]
Output: [3, 7, 5, 2, 9]
Input: [-1, 2, -1, 3, 2, -2]
Output: [-1, 2, 3, -2]
Input: [1000000, 999999, 1000000]
Output: [1000000, 999999]

>>>
= RESTART: C:/Users/HP/AppData/Local/Programs/Python/Python313/prpogram 8 bubble sort.py
Input: [5, 2, 9, 1, 5, 6]
Output: [1, 2, 5, 5, 6, 9]
Input: [3, 7, 4, 2, 8]
Output: [2, 3, 4, 7, 8]
Input: [10, -2, 0, 5, 3]
Output: [-2, 0, 3, 5, 10]
```

9. Sort the array by using binary search :

```
program 9 binary search.py - C:/Users/HP/OneDrive/Desktop/DAA lab programmes/program 9 binary search.py
File Edit Format Run Options Window Help
def binary_search(arr, key):
    arr.sort() # Ensure the array is sorted
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == key:
            return mid # Element found
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1

    return -1 # Element not found

# Test Cases
# Test Case 1
arr1 = [3, 4, 6, -9, 10, 8, 9, 30]
key1 = 10
pos1 = binary_search(arr1, key1)
if pos1 != -1:
    print(f"Output: Element {key1} is found at position {pos1}")
else:
    print(f"Output: Element {key1} is not found")

# Test Case 2
arr2 = [3, 4, 6, -9, 10, 8, 9, 30]
key2 = 100
pos2 = binary_search(arr2, key2)
if pos2 != -1:
    print(f"Output: Element {key2} is found at position {pos2}")
else:
    print(f"Output: Element {key2} is not found")

Ln 36 Col 0
```

```
IDLE Shell 3.13.5
File Edit Shell Debug Options Window Help
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.

>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programmes/program 6 sort and find the maximum.py
Input: []
Output: List is empty
Input: [5]
Output: 5
Input: [3, 3, 3, 3]
Output: 3

>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programmes/program 7 remove the duplicate s.py
Input: [3, 7, 3, 5, 2, 5, 9, 2]
Output: [3, 7, 5, 2, 9]
Input: [-1, 2, -1, 3, 2, -2]
Output: [-1, 2, 3, -2]
Input: [1000000, 999999, 1000000]
Output: [1000000, 999999]

>>>
= RESTART: C:/Users/HP/AppData/Local/Programs/Python/Python313/prpogram 8 bubble sort.p
Input: [5, 2, 9, 1, 5, 6]
Output: [1, 2, 5, 5, 6, 9]
Input: [3, 7, 4, 2, 8]
Output: [2, 3, 4, 7, 8]
Input: [10, -2, 0, 5, 3]
Output: [-2, 0, 3, 5, 10]

>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programmes/program 9 binary search.py =
Output: Element 10 is found at position 6
Output: Element 100 is not found

>>>
```

10. Sort the array by using merge sort :

```
program 10 merge sort.py - C:/Users/HP/OneDrive/Desktop/DAA lab programmes/program 10 merge sort.py
File Edit Format Run Options Window Help
def merge_sort(nums):
    if len(nums) <= 1:
        return nums

    mid = len(nums) // 2
    left = merge_sort(nums[:mid])
    right = merge_sort(nums[mid:])

    # Merge two sorted halves
    merged = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            j += 1

    # Append remaining elements
    merged.extend(left[i:])
    merged.extend(right[j:])

    return merged

# Test Cases
# Test Case 1
nums1 = [5, 2, 9, 1, 5, 6]
print("Input:", nums1)
print("Output:", merge_sort(nums1))

# Test Case 2
nums2 = [3, 7, 4, 2, 8]
print("Input:", nums2)
print("Output:", merge_sort(nums2))

# Test Case 3
nums3 = [10, -2, 0, 5, 3]
print("Input:", nums3)
print("Output:", merge_sort(nums3))

Ln 42 Col 0
```

```
IDLE Shell 3.13.5
File Edit Shell Debug Options Window Help
Python 3.13.5 (tags/v3.13.5:6cb20a2, Jun 11 2025, 16:15:46) [MSC v.1943 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.

>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programmes/program 6 sort and find the maximum.py
Input: []
Output: List is empty
Input: [5]
Output: 5
Input: [3, 3, 3, 3]
Output: 3

>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programmes/program 7 remove the duplicate s.py
Input: [3, 7, 3, 5, 2, 5, 9, 2]
Output: [3, 7, 5, 2, 9]
Input: [-1, 2, -1, 3, 2, -2]
Output: [-1, 2, 3, -2]
Input: [1000000, 999999, 1000000]
Output: [1000000, 999999]

>>>
= RESTART: C:/Users/HP/AppData/Local/Programs/Python/Python313/prpogram 8 bubble sort.p
Input: [5, 2, 9, 1, 5, 6]
Output: [1, 2, 5, 5, 6, 9]
Input: [3, 7, 4, 2, 8]
Output: [2, 3, 4, 7, 8]
Input: [10, -2, 0, 5, 3]
Output: [-2, 0, 3, 5, 10]

>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programmes/program 9 binary search.py =
Output: Element 10 is found at position 6
Output: Element 100 is not found

>>>
= RESTART: C:/Users/HP/OneDrive/Desktop/DAA lab programmes/program 10 merge sort.py ==
Input: [5, 2, 9, 1, 5, 6]
Output: [1, 2, 5, 5, 6, 9]
Input: [3, 7, 4, 2, 8]
Output: [2, 3, 4, 7, 8]
Input: [10, -2, 0, 5, 3]
Output: [-2, 0, 3, 5, 10]

>>>
```

